

Lecture 10

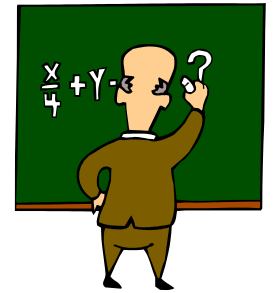
Dynamic Programming



Learning Outcome

- ◆ The General Technique
- ◆ 0-1 Knapsack Problem
- ◆ Transitive closure
 - The Floyd-Warshall Algorithm

The General Dynamic Programming Technique



- ◆ Applies to a problem that at first seems to require a lot of time (possibly exponential), provided we have:
- **Simple subproblems:** the subproblems can be defined in terms of a few variables, such as j, k, l, m , and so on.
 - **Subproblem optimality:** the global optimum value can be defined in terms of optimal subproblems
 - **Subproblem overlap:** the subproblems are not independent, but instead they overlap (hence, should be constructed bottom-up).

The 0/1 Knapsack Problem



- ◆ Given: A set S of n items, with each item i having
 - b_i - a positive benefit
 - w_i - a positive weight
- ◆ Goal: Choose items with maximum total benefit but with weight at most W .
- ◆ If we are **not allowed** to take fractional amounts, then this is the **0/1 knapsack problem**.
 - In this case, we let T denote the set of items we take

- Objective: maximize
$$\sum_{i \in T} b_i$$

- Constraint:
$$\sum_{i \in T} w_i \leq W$$

A 0/1 Knapsack Algorithm, First Attempt



- ◆ Greedy: repeatedly add item with maximum ratio, benefit/ weight
- ◆ Problem: greedy method does not have subproblem optimality
- ◆ $W = 11$ kg
- ◆ $\{5, 2, 1\}$ has benefit $= 28 + 6 + 1 = 35$, greedy not optimal
- ◆ $\{3, 4\}$ has optimum benefit of 40 ($18 + 22$)!

Items:	1	2	3	4	5
Weight (kg):	1	2	5	6	7
Benefit:	1	6	18	22	28
Value:	1	3	3.6	3.67	4

A 0/1 Knapsack Algorithm, Second Attempt



- ◆ S_k : Set of items numbered 1 to k .
- ◆ Define $B[k, w] = \text{best selection from } S_k \text{ with weight at most } w$
- ◆ Good news: this does have subproblem optimality:

$$B[k, w] = \begin{cases} B[k-1, w] & \text{if } w_k > w \\ \max\{B[k-1, w], B[k-1, w - w_k] + b_k\} & \text{else} \end{cases}$$

- ◆ i.e., best subset of S_k with weight exactly w is either the best subset of S_{k-1} w/ weight w or the best subset of S_{k-1} w/ weight $w - w_k$ plus item k .

The 0/1 Knapsack Algorithm



- ◆ Recall definition of $B[k, w]$:

$$B[k, w] = \begin{cases} B[k-1, w] & \text{if } w_k > w \\ \max\{B[k-1, w], B[k-1, w - w_k] + b_k\} & \text{else} \end{cases}$$

- ◆ Since $B[k, w]$ is defined in terms of $B[k-1, *]$, we can reuse the same array
- ◆ Running time: $O(nW)$
- ◆ Not a polynomial-time algorithm if W is large
- ◆ This is a pseudo-polynomial time algorithm

Algorithm **01Knapsack**(S, W):

Input: set S of items w/ benefit b_i and weight w_i ; max. weight W

Output: benefit of best subset with weight at most W

```
for  $w = 0$  to  $W$ 
     $B[w] = 0$ 
for  $k = 1$  to  $n$ 
    for  $w = W$  downto  $w_k$ 
        if  $B[w - w_k] + b_k > B[w]$ 
             $B[w] = B[w - w_k] + b_k$ 
```

The 0/1 Knapsack Algorithm

W

k

	0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1{1}	1	1	1	1	1	1	1	1	1	1
2	0	1	6{2}	7{1,2}	7	7	7	7	7	7	7	7
3	0	1	6	7	7	18{3}	19{1,3}	24{2,3}	25{1-3}	25	25	25
4	0	1	6	7	7	18	22{4}	24{2,3}	28{2,4}	29{1,2,6}	29	40{3,4}
5	0	1	6	7	7	18	22{4}	28{5}	29{1,7}	34{2,5}	35{1,2,5}	40

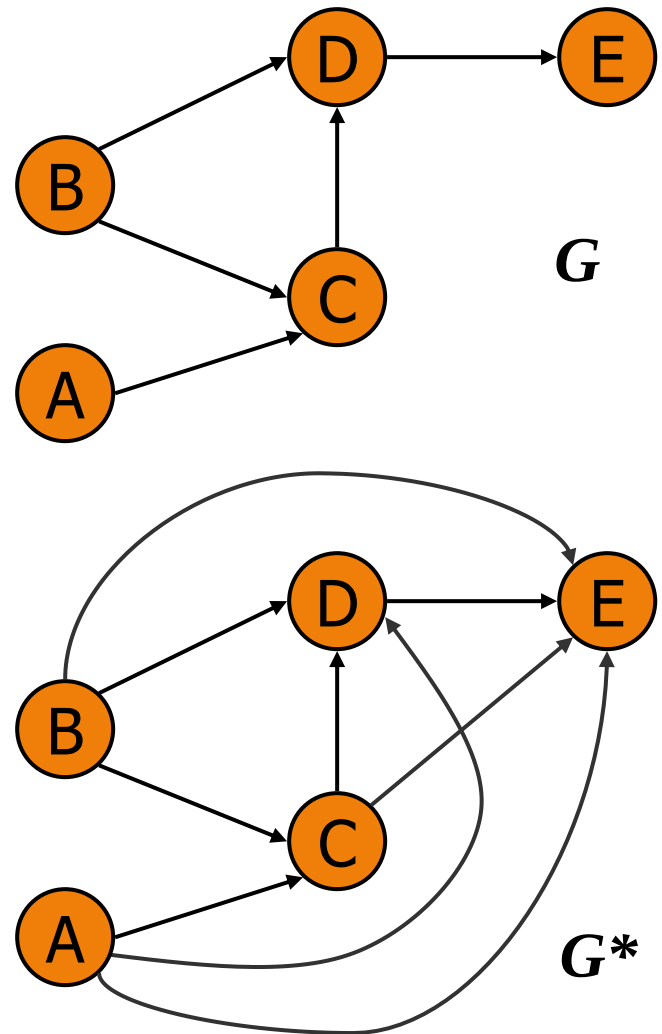
Items: 1 2 3 4 5

Weight (kg): 1 2 5 6 7

Benefit: 1 6 18 22 28

Transitive Closure Problem

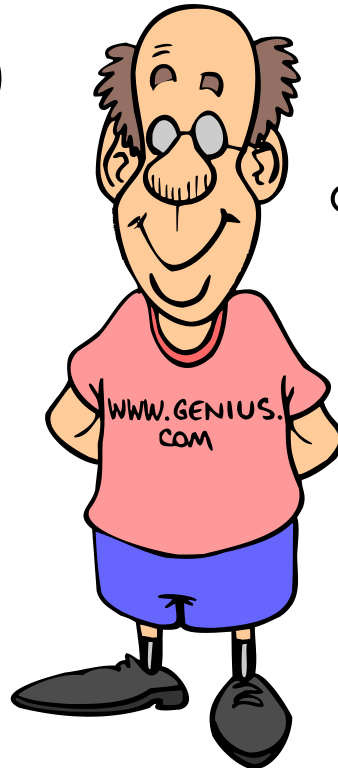
- ◆ Given a digraph G , the transitive closure of G is the digraph G^* such that
 - G^* has the same vertices as G
 - if G has a directed path from u to v ($u \neq v$), G^* has a directed edge from u to v
- ◆ The transitive closure provides reachability information about a digraph



Computing the Transitive Closure

- ◆ We can perform DFS starting at each vertex

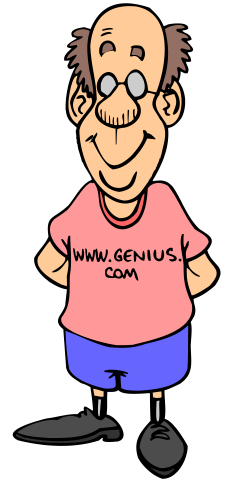
- $O(n(n+m))$



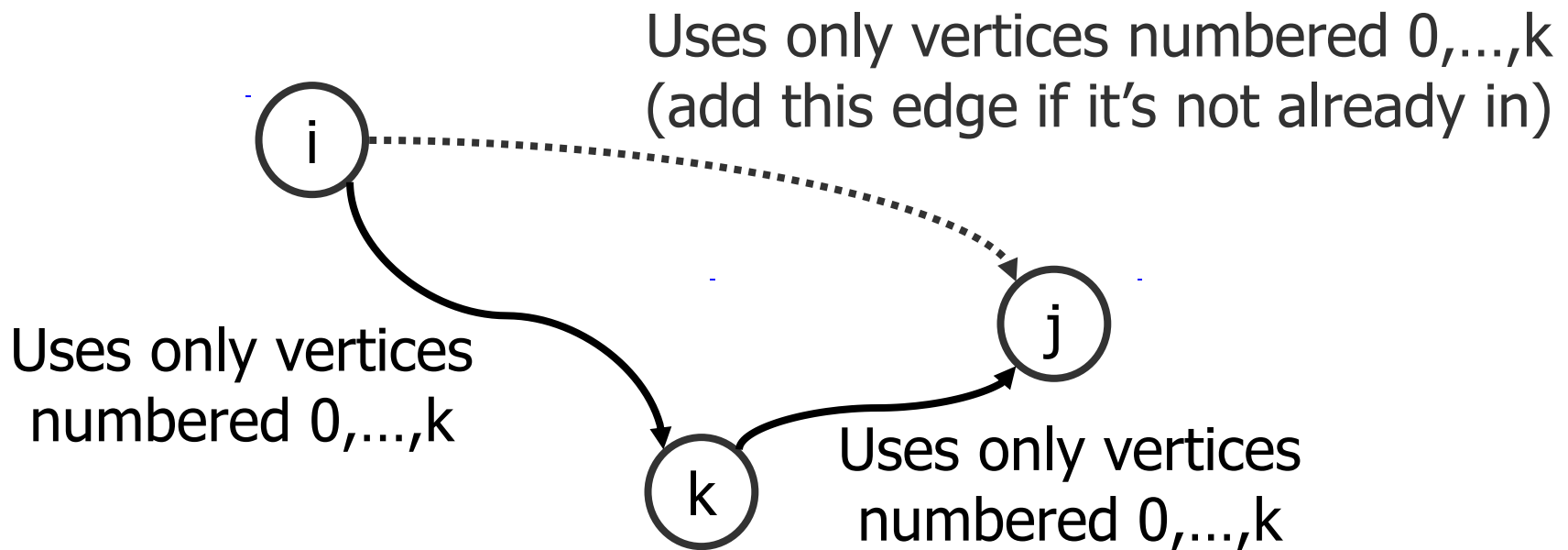
If there's a way to get from **A** to **B** and from **B** to **C**, then there's a way to get from **A** to **C**.

- ◆ Alternatively ... Use dynamic programming:
The Floyd-Warshall Algorithm

Floyd-Warshall Transitive Closure



- ◆ Idea #1: Number the vertices $0, 1, \dots, n-1$.
- ◆ Idea #2: Consider paths that use only vertices numbered $0, 1, \dots, k$, as intermediate vertices:





Floyd-Warshall's Algorithm

- ◆ Floyd-Warshall's algorithm numbers the vertices of G as $v_0, \dots, v_{n-1}$ and computes a series of digraphs $G_0, \dots, G_{n-1}$
 - G_k has a directed edge (v_i, v_j) if G has a directed path from v_i to v_j with intermediate vertices in the set $\{v_0, \dots, v_k\}$
- ◆ We have that $G_{n-1} = G^*$
- ◆ In phase k , digraph G_k is computed from G_{k-1}
- ◆ Running time: $O(n^3)$, assuming `areAdjacent` is $O(1)$ (e.g., adjacency matrix)

Algorithm FloydWarshall(G)

Input: A **digraph** G

Output: A **transitive closure** G^* of G

$G_0 = G$

for $k = 0$ to $n-1$

 if $k > 0$

$G_k = G_{k-1}$

 for $i = 0$ to $n-1$ ($i \neq k$)

 for $j = 0$ to $n-1$ ($j \neq i, k$)

 if $G_{k-1}.\text{areAdjacent}(v_i, v_k)$

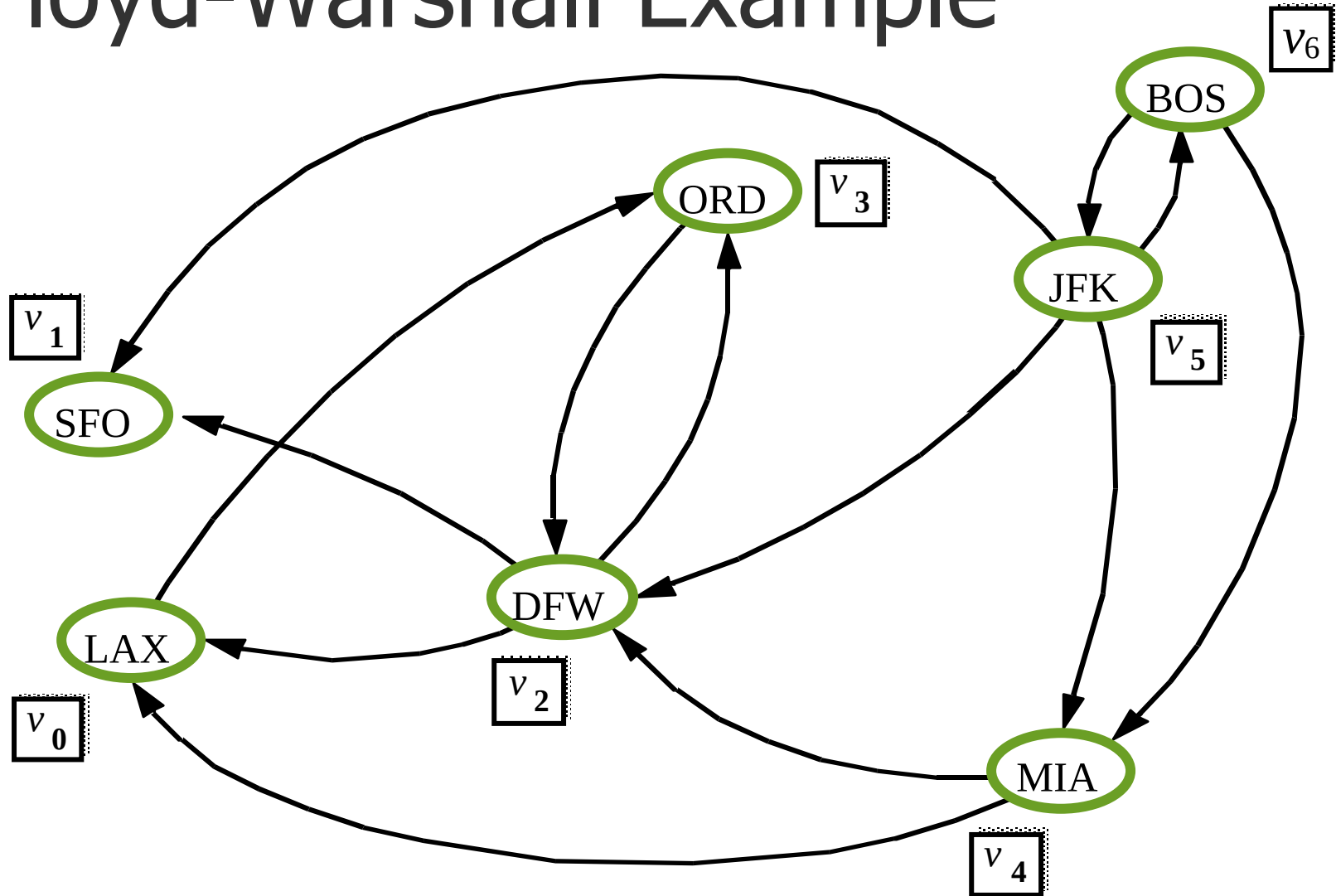
 and $G_{k-1}.\text{areAdjacent}(v_k, v_j)$

 if not $G_k.\text{areAdjacent}(v_i, v_j)$

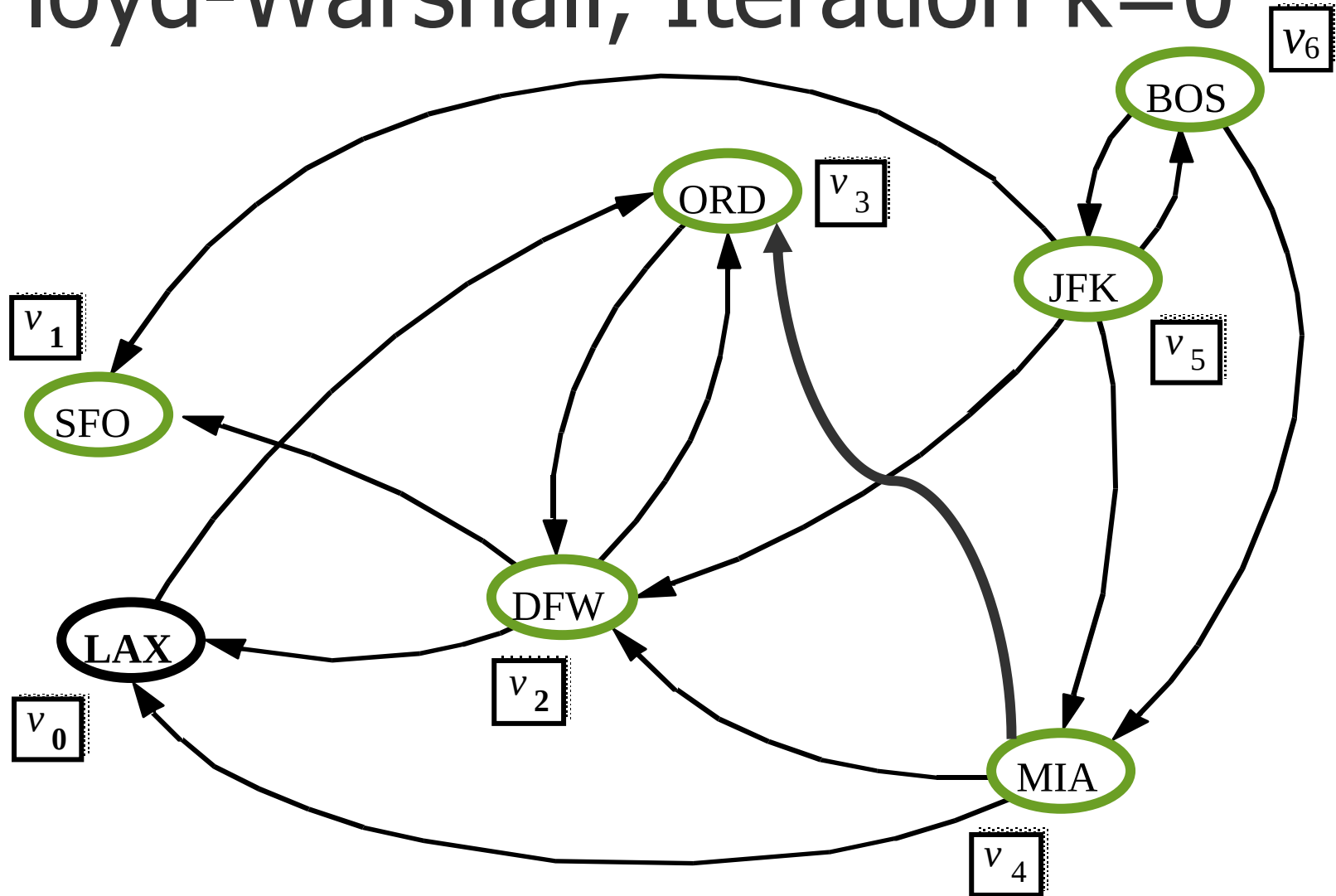
$G_k.\text{insertEdge}(v_i, v_j)$

 return G_n

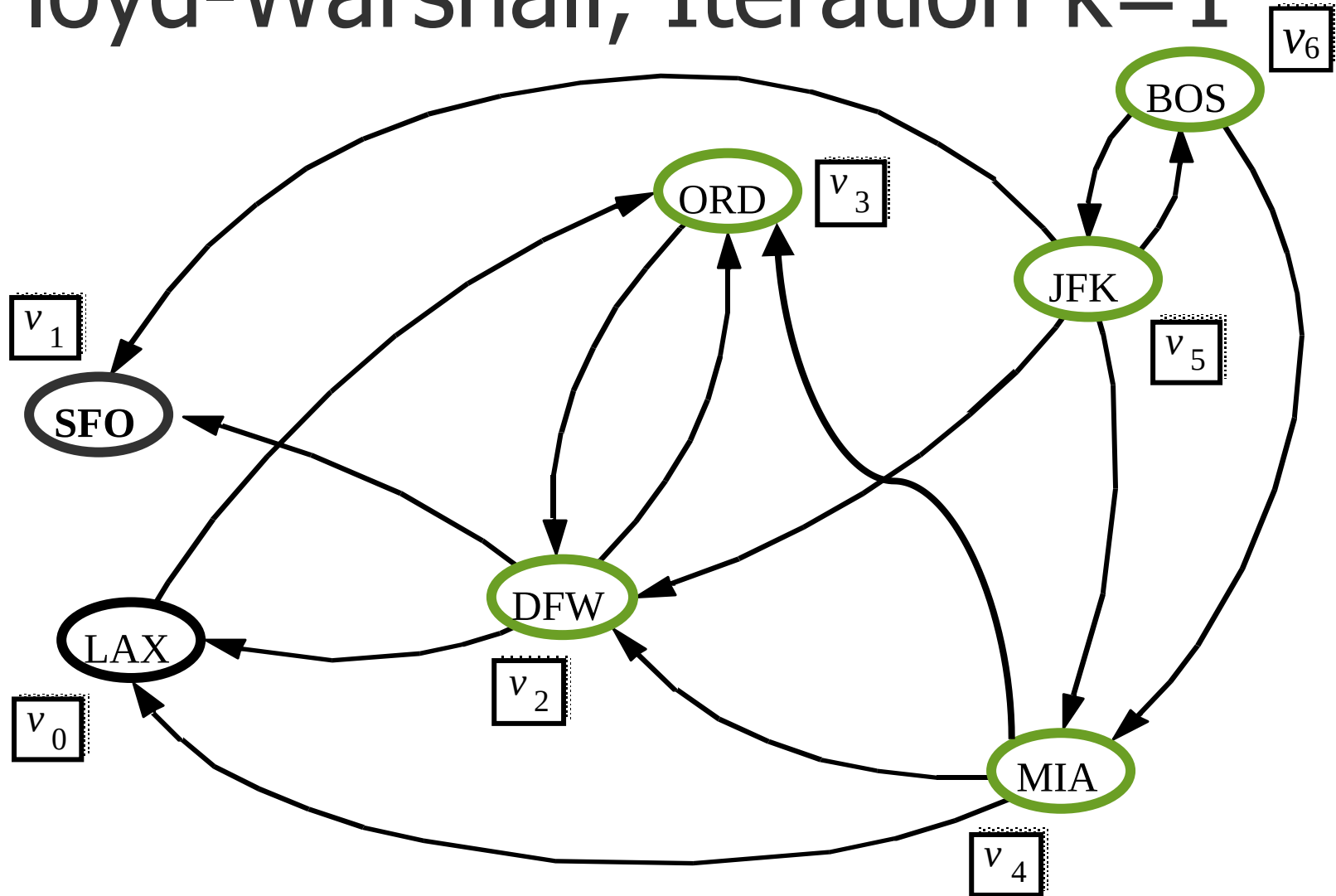
Floyd-Warshall Example



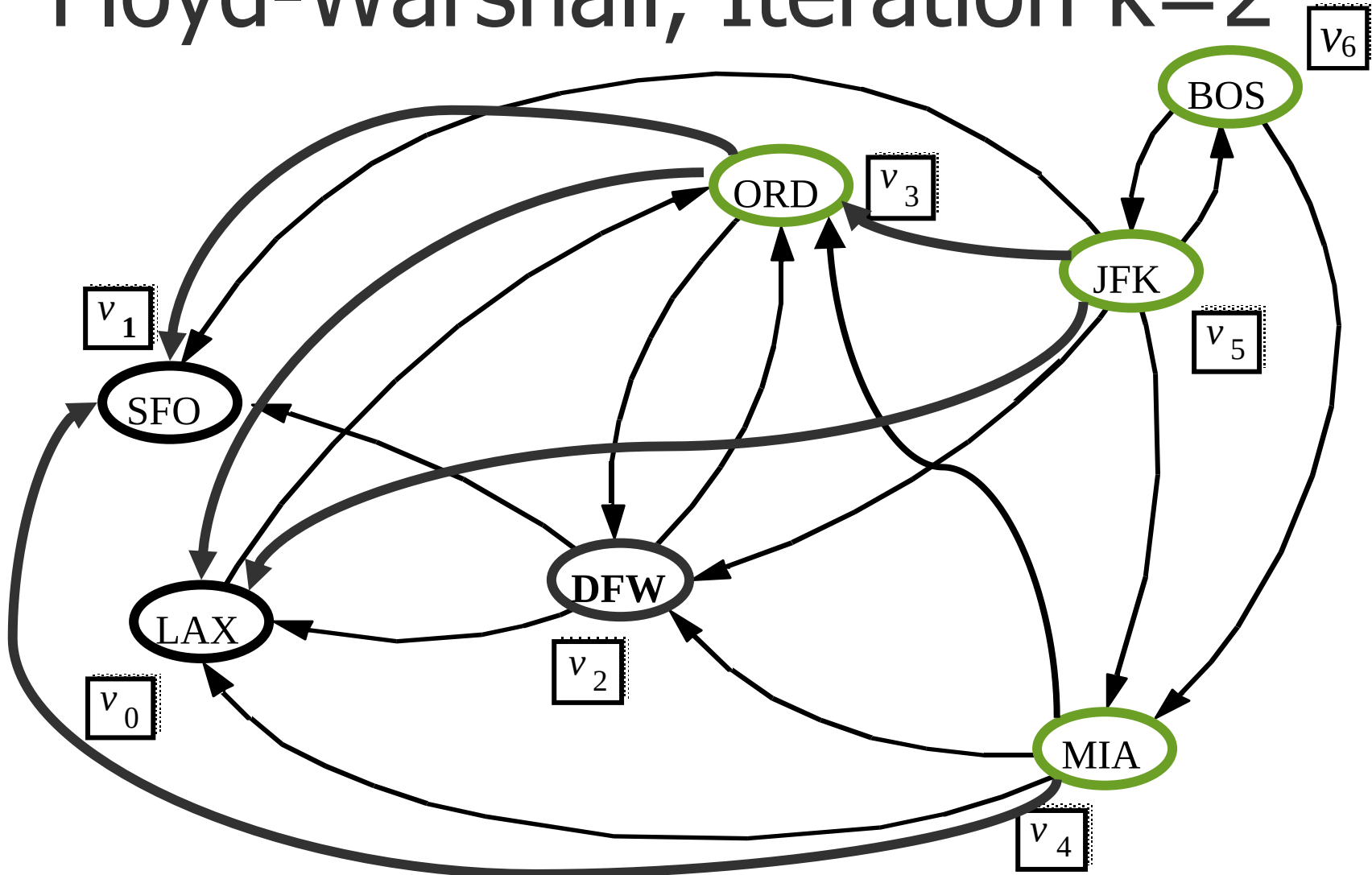
Floyd-Warshall, Iteration k=0



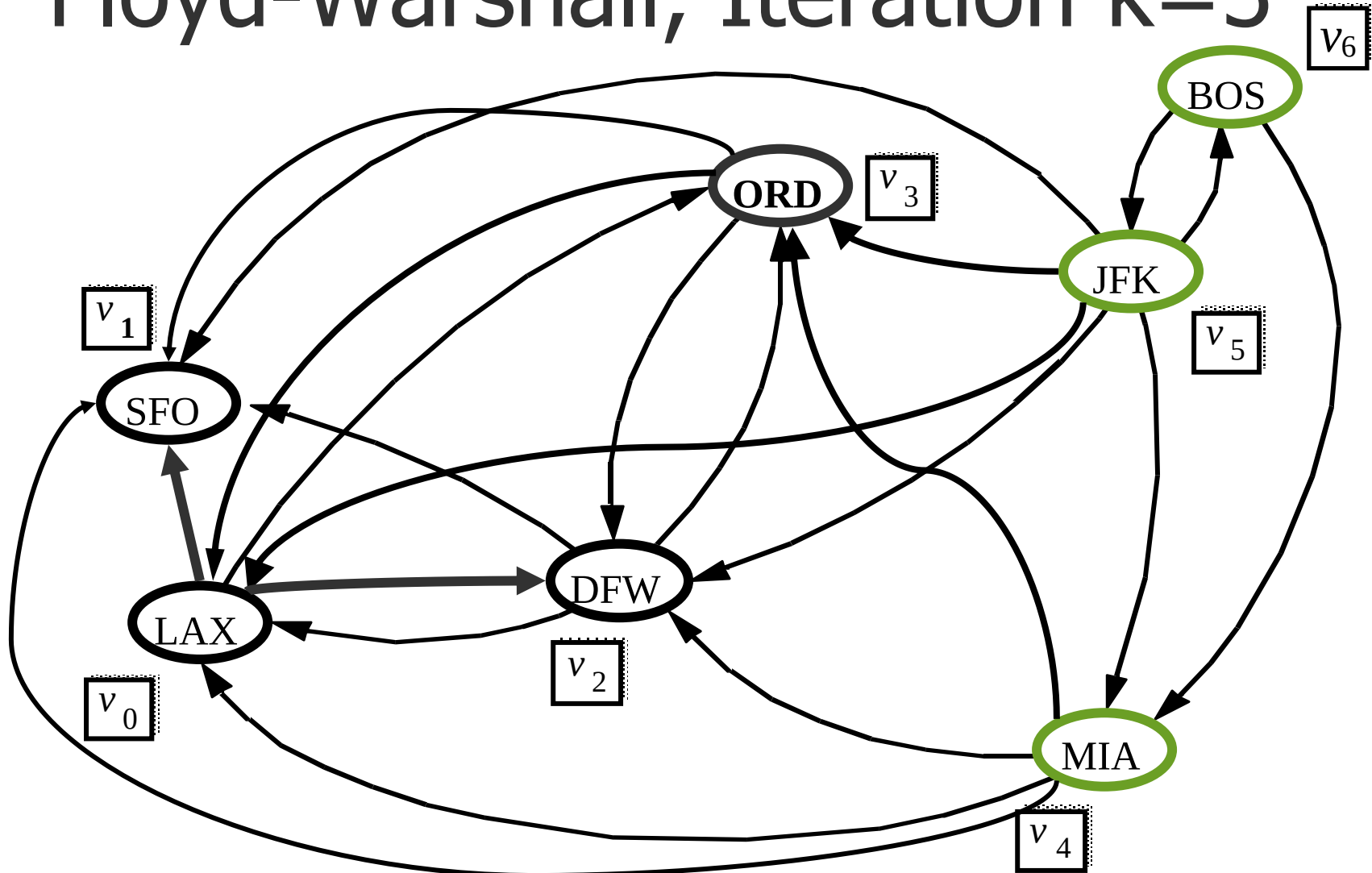
Floyd-Warshall, Iteration k=1



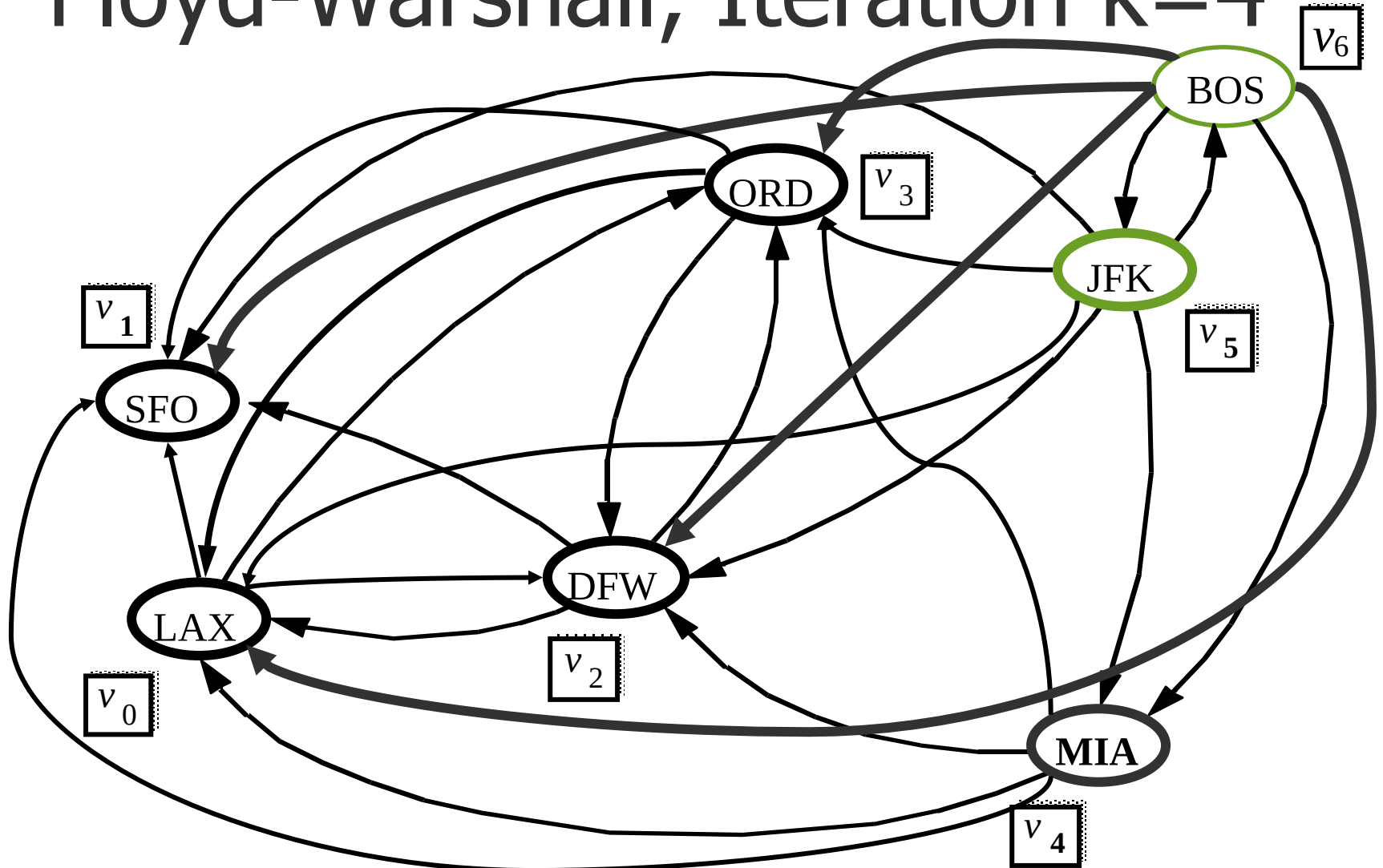
Floyd-Warshall, Iteration k=2



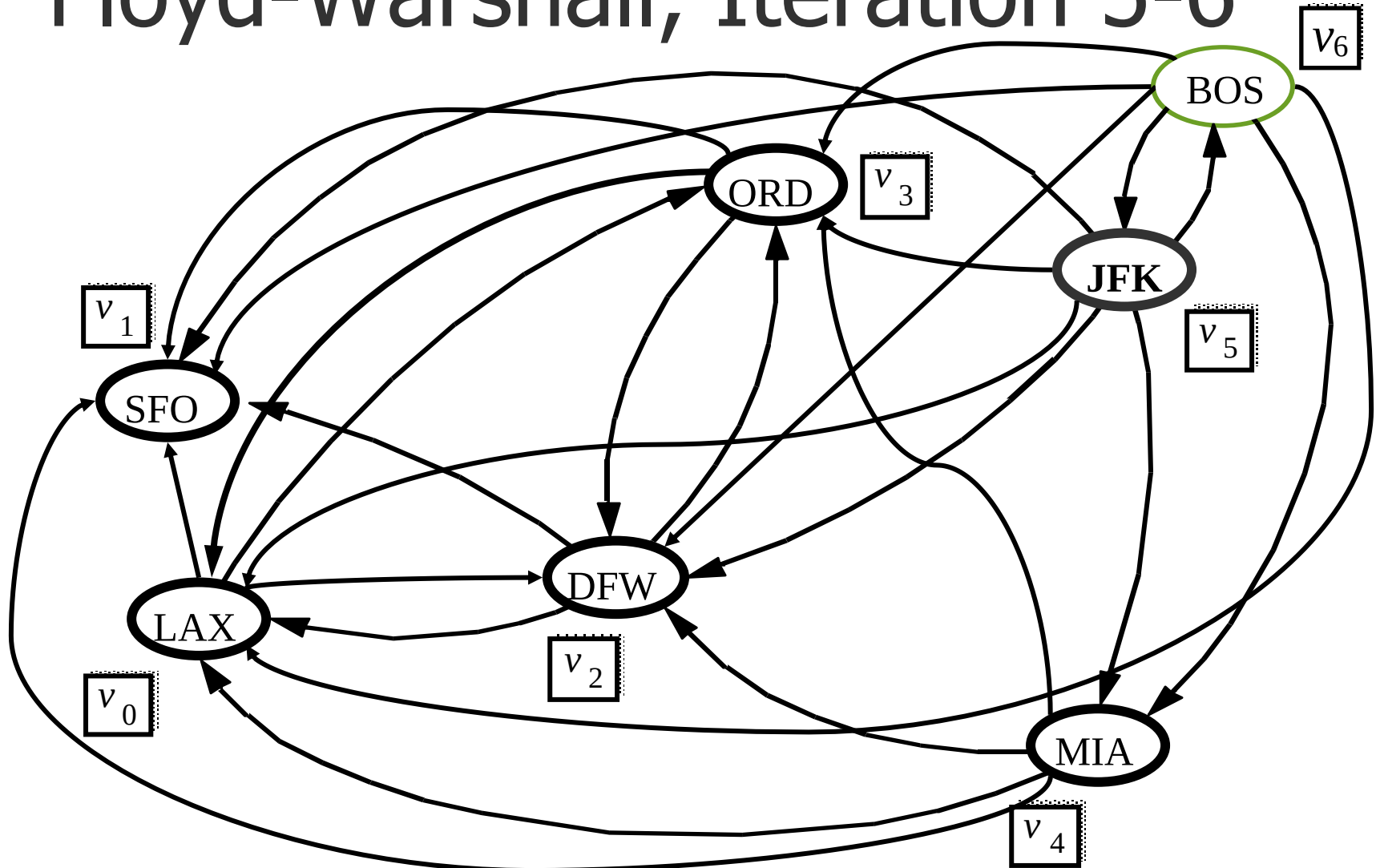
Floyd-Warshall, Iteration k=3



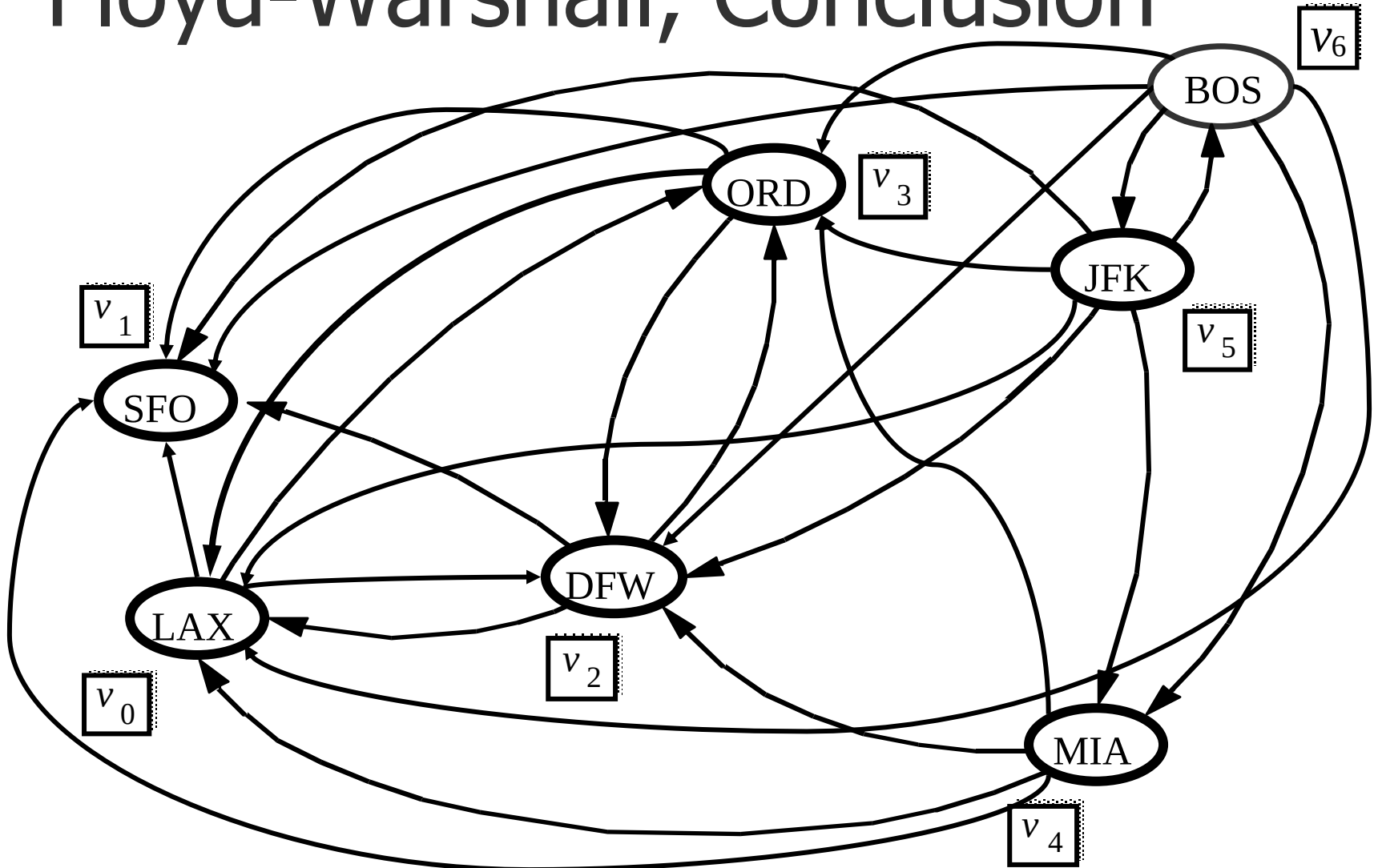
Floyd-Warshall, Iteration k=4



Floyd-Warshall, Iteration 5-6



Floyd-Warshall, Conclusion



Floyd-Warshall Example

◆ Adjacency matrix

	0	1	2	3	4	5	6

0	0	0	0	1	0	0	0
1	0	0	0	0	0	0	0
2	1	1	0	1	0	0	0
3	0	0	1	0	0	0	0
4	1	0	1	0	0	0	0
5	0	1	1	0	1	0	1
6	0	0	0	0	1	1	0

Floyd-Warshall, Iteration k=0

- if $G_{k-1}.\text{areAdjacent}(v_i, \emptyset)$ and $G_{k-1}.\text{areAdjacent}(\emptyset, v_j)$

	<u>0</u>	1	2	3	4	5	6

<u>0</u>	0	∞	∞	<u>1</u>	∞	∞	∞
1	∞	0	∞	∞	∞	∞	∞
2	<u>1</u>	1	0	1	∞	∞	∞
3	∞	∞	1	0	∞	∞	∞
4	<u>1</u>	∞	1	∞	0	∞	∞
5	∞	1	1	∞	1	0	1
6	∞	∞	∞	∞	1	1	0

k = 0

	0	1	2	3	4	5	6

0	0	∞	∞	1	∞	∞	∞
1	∞	0	∞	∞	∞	∞	∞
2	1	1	0	1	∞	∞	∞
3	∞	∞	1	0	∞	∞	∞
4	1	∞	1	1	0	∞	∞
5	∞	1	1	∞	1	0	1
6	∞	∞	∞	∞	1	1	0

Floyd-Warshall, Iteration k=1

- if $G_{k-1}.\text{areAdjacent}(v_i, 1)$ and $G_{k-1}.\text{areAdjacent}(1, v_j)$

k = 0

0 1 2 3 4 5 6

0	0	∞	∞	1	∞	∞	∞
<u>1</u>	∞	0	∞	∞	∞	∞	∞
2	1	<u>1</u>	0	1	∞	∞	∞
3	∞	∞	1	0	∞	∞	∞
4	1	∞	1	1	0	∞	∞
5	∞	<u>1</u>	1	∞	1	0	1
6	∞	∞	∞	∞	1	1	0

k = 1 (no change)

0 1 2 3 4 5 6

0	0	∞	∞	1	∞	∞	∞
1	∞	0	∞	∞	∞	∞	∞
2	1	1	0	1	∞	∞	∞
3	∞	∞	1	0	∞	∞	∞
4	1	∞	1	1	0	∞	∞
5	∞	1	1	∞	1	0	1
6	∞	∞	∞	∞	1	1	0

Floyd-Warshall, Iteration k=2

- if $G_{k-1}.\text{areAdjacent}(v_i, 2)$ and $G_{k-1}.\text{areAdjacent}(2, v_j)$

k = 1

	0	1	<u>2</u>	3	4	5	6

0	0	∞	∞	1	∞	∞	∞
1	∞	0	∞	∞	∞	∞	∞
<u>2</u>	<u>1</u>	<u>1</u>	0	<u>1</u>	∞	∞	∞
3	∞	∞	<u>1</u>	0	∞	∞	∞
4	1	∞	<u>1</u>	1	0	∞	∞
5	∞	1	<u>1</u>	∞	1	0	1
6	∞	∞	∞	∞	1	1	0

k = 2

	0	1	2	3	4	5	6

0	0	∞	∞	1	∞	∞	∞
1	∞	0	∞	∞	∞	∞	∞
2	1	1	0	1	∞	∞	∞
3	1	1	1	0	∞	∞	∞
4	1	1	1	1	0	∞	∞
5	1	1	1	1	1	0	1
6	∞	∞	∞	∞	1	1	0

Floyd-Warshall, Iteration k=3

- if $G_{k-1}.\text{areAdjacent}(v_i, 3)$ and $G_{k-1}.\text{areAdjacent}(3, v_j)$

k = 2

	0	1	2	<u>3</u>	4	5	6

0	0	∞	∞	<u>1</u>	∞	∞	∞
1	∞	0	∞	∞	∞	∞	∞
2	1	1	0	<u>1</u>	∞	∞	∞
<u>3 </u>	<u>1</u>	<u>1</u>	<u>1</u>	0	∞	∞	∞
4	1	1	1	<u>1</u>	0	∞	∞
5	1	1	1	<u>1</u>	1	0	1
6	∞	∞	∞	∞	1	1	0

k = 3

	0	1	2	3	4	5	6

0	0	1	1	1	∞	∞	∞
1	∞	0	∞	∞	∞	∞	∞
2	1	1	0	1	∞	∞	∞
3	1	1	1	0	∞	∞	∞
4	1	1	1	1	0	∞	∞
5	1	1	1	1	1	0	1
6	∞	∞	∞	∞	1	1	0

Floyd-Warshall, Iteration k=4

- if $G_{k-1}.\text{areAdjacent}(v_i, 4)$ and $G_{k-1}.\text{areAdjacent}(4, v_j)$

k = 3

	0	1	2	<u>3</u>	4	5	6

0	0	1	1	1	∞	∞	∞
1	∞	0	∞	∞	∞	∞	∞
2	1	1	0	1	∞	∞	∞
<u>3 </u>	<u>1</u>	<u>1</u>	<u>1</u>	0	∞	∞	∞
4	1	1	1	1	0	∞	∞
5	1	1	1	1	<u>1</u>	0	1
6	∞	∞	∞	∞	<u>1</u>	1	0

k = 4

	0	1	2	3	4	5	6

0	0	1	1	1	∞	∞	∞
1	∞	0	∞	∞	∞	∞	∞
2	1	1	0	1	∞	∞	∞
3	1	1	1	0	∞	∞	∞
4	1	1	1	1	0	∞	∞
5	1	1	1	1	1	0	1
6	1	1	1	1	1	1	0

Floyd-Warshall, Iteration k=5

- if $G_{k-1}.\text{areAdjacent}(v_i, 5)$ and $G_{k-1}.\text{areAdjacent}(5, v_j)$

k = 4

	0	1	2	3	<u>4</u>	5	6

0	0	1	1	1	∞	∞	∞
1	∞	0	∞	∞	∞	∞	∞
2	1	1	0	1	∞	∞	∞
3	1	1	1	0	∞	∞	∞
<u>4</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	0	∞	∞
5	1	1	1	1	<u>1</u>	0	1
6	1	1	1	1	<u>1</u>	1	0

k = 5 (no change)

	0	1	2	3	4	5	6

0	0	1	1	1	∞	∞	∞
1	∞	0	∞	∞	∞	∞	∞
2	1	1	0	1	∞	∞	∞
3	1	1	1	0	∞	∞	∞
4	1	1	1	1	0	∞	∞
5	1	1	1	1	1	0	1
6	1	1	1	1	1	1	0

Floyd-Warshall, Iteration k=6

- if $G_{k-1}.\text{areAdjacent}(v_i, 6)$ and $G_{k-1}.\text{areAdjacent}(v_k, 6)$

k = 5

	0	1	2	3	4	<u>5</u>	6

0	0	1	1	1	∞	∞	∞
1	∞	0	∞	∞	∞	∞	∞
2	1	1	0	1	∞	∞	∞
3	1	1	1	0	∞	∞	∞
4	1	1	1	1	0	∞	∞
<u>5 </u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	<u>1</u>	0	<u>1</u>
6	1	1	1	1	1	<u>1</u>	0

k = 6 (no change)

	0	1	2	3	4	5	6

0	0	1	1	1	∞	∞	∞
1	∞	0	∞	∞	∞	∞	∞
2	1	1	0	1	∞	∞	∞
3	1	1	1	0	∞	∞	∞
4	1	1	1	1	0	∞	∞
5	1	1	1	1	1	0	1
6	1	1	1	1	1	1	0